CUDA Kernel 提交开销

引言

通过 Profiling 可以发现单个 Kernel 执行所需时间，除了 CPU 上 kernel launch 的时间和 kernel 实际执行时间，还包括了 kernel 提交到设备上初始化参数配置和执行环境的时间。这个阶段即有时间上的开销，也会存在一定的数据流量。

Kernel 提交的时间开销

GPU 架构的性能随着每一代的更新而不断提高。 现代 GPU 每个操作（如 kernel 运行或内存复制）所花费的时间现在以微秒为单位。 但是，将每个操作提交给 GPU 也会产生一些开销——也是微秒级的。

实际的应用程序中经常要执行大量的 GPU 操作：典型模式涉及许多迭代，每个步骤中有多个操作。如果这些操作中的每一个都单独提交到 GPU 启动并独立计算，特别是当 kernel 实际执行时间比较短时，kernel launch 和 kernel 提交到设备的时间占比较多，那么提交启动开销汇总在一起可能导致明显的整体性能下降。

考虑如下的一种简单的情况，sleep_us kernel 会 sleep 100 us，外层循环 5 次，表示多次推理流程；内层循环调用 1000 次 sleep_us，表示一个 step 中调用的模型的各个 kernels。

sleep_us C++

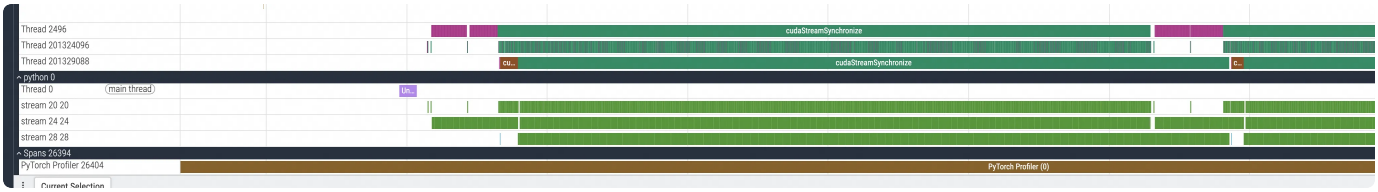
```cpp
__global__ void sleep_us_kernel(uint64_t sleep_cycles)
{
    uint64_t start_cycle = clock64();
    while ((clock64() - start_cycle) < sleep_cycles)
        ;
}

void sleep_us(uint64_t time_us, cudaStream_t stream)
{
    // 使用静态变量缓存设备属性，避免每次调用都查询
    static uint64_t cached_sleep_cycles = 0;
    if (cached_sleep_cycles == 0) {
        cudaDeviceProp prop;
        cudaGetDeviceProperties(&prop, 0);
        uint64_t cycles_per_second = prop.clockRate * 1000;
        cached_sleep_cycles = time_us * cycles_per_second / 1000000ULL;
    }
    sleep_us_kernel<<<1, 1, 0, stream>>>(cached_sleep_cycles);
}
```

依次采用三种方案执行以上流程：

1. 两层循环，没执行一次 sleep_us 后都通过 `cudaStreamSynchronize` 同步 stream；
2. 两层循环，只在内层循环结束后同步 stream；
3. 只有外层循环，内部的 1000 次执行采用 CUDA Graph；

这三种方案的执行情况 timeline 如下图所示：

📎kernel_commit_comparison.json



方案 1/2/3 对应 stream 20/24/28

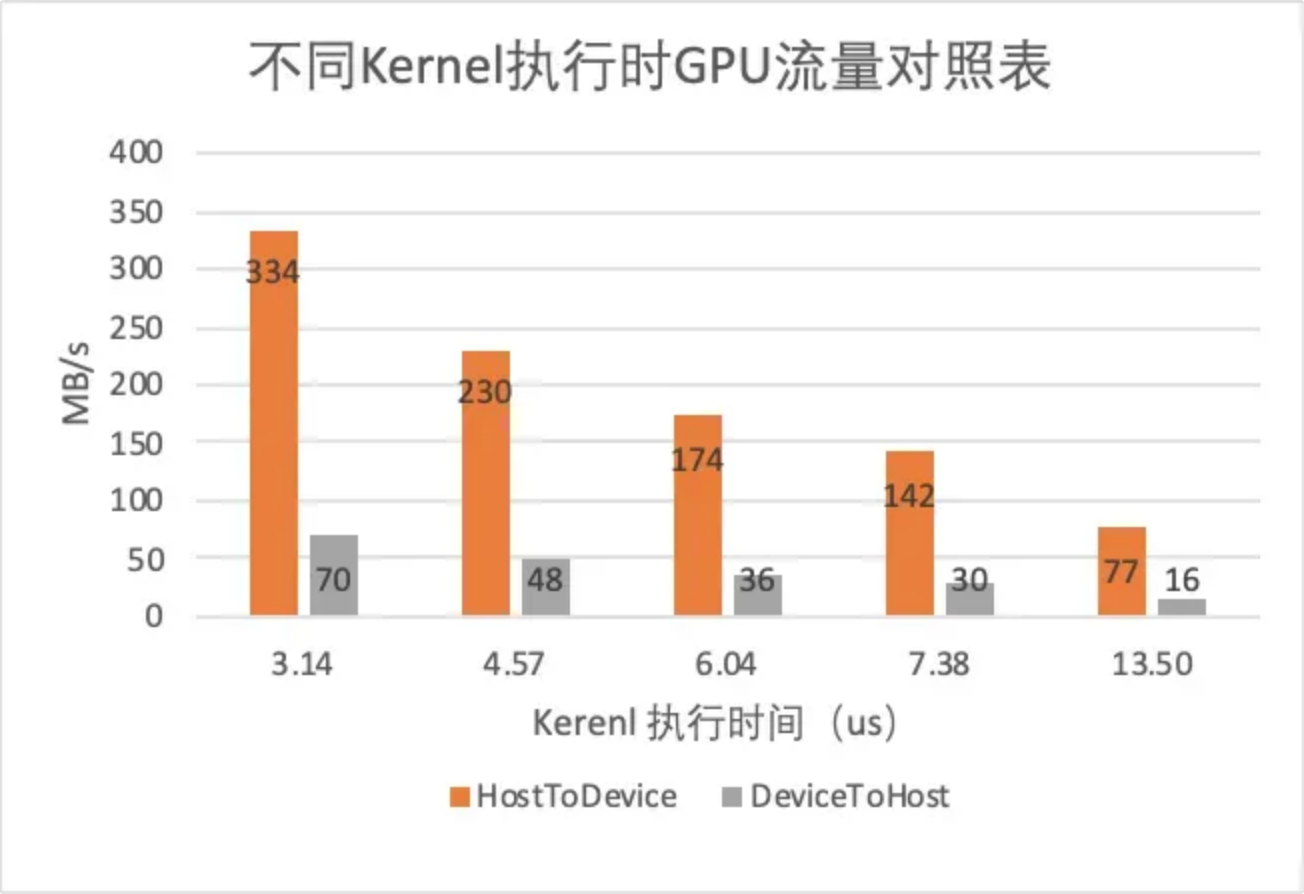
可以看出每次调用 sleep_us 后都同步 stream 会导致 kernels 间的空闲间隔比较大，这段区间包含了 **CPU 上循环等执行耗时**、**kernel_launch 耗时**以及**kernel 提交到设备上的耗时**。

当把同步 stream 移到外层循环后，内层循环相邻 sleep_us 之间的空闲间隔明显缩短，这是因为 **CPU 和 GPU 之间的异步执行使得 CPU 上操作的耗时都隐藏在 kernel 的执行耗时之后**，这使得 kernel 之间的空闲间隔只包括 kernel 提交到设备上的耗时。还是可以发现当前方案相对于 CUDA Graph 方案的空闲间隔还是会长一些。

CUDA Graph 方案在**实例化可执行图时**、**单独调用** `cudaGraphUpload` 或者**首次执行 graph** 时均可以将 kernel 提交到设备上 (主要映射物理内存并初始化执行环境)，之后执行 graph 都不需要再执行该操作。而单独启动 kernel 每次都会重新将 kernel 提交到设备上，因此 CUDA Graph 可以进一步优化性能。

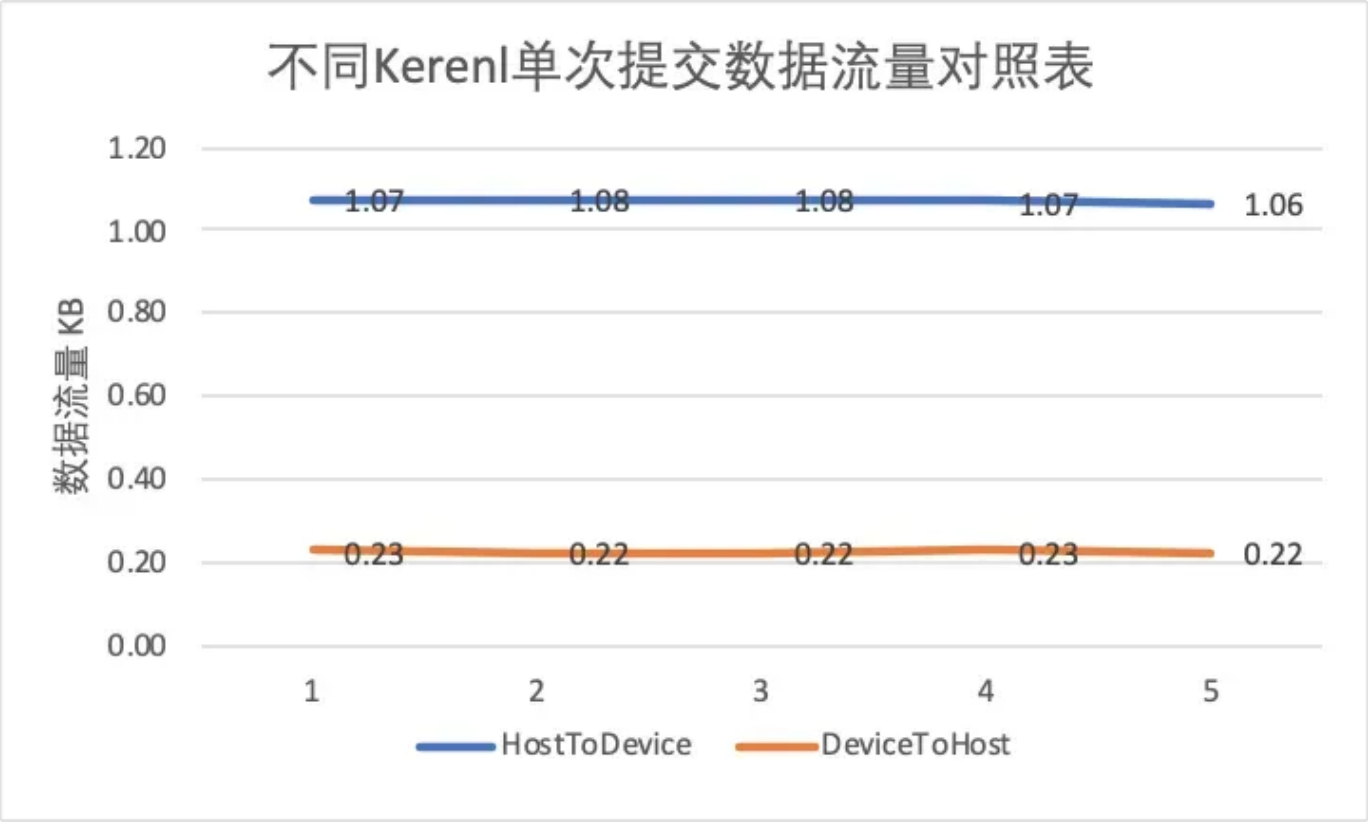
Kernel 提交的流量开销

在 Kernel 的执行过程中是有持续的双向数据流量的，即使 Kernel 执行过程中没有运算数据流量，`HostToDevice` 的流量就是 CPU 侧 Kernel 提交产生的，`DeviceToHost` 的流量就是 GPU 执行完 Kernel 需要和 Host 交互产生的。而且这个流量是和 Kernel 的提交频率有关，和 Kernel 本身并没有关系。



不同执行时间 Kernel 对应的GPU流量对照图

进一步研究 Kernel 提交究竟会有多大的数据流量，下图做了一个展示，单次 Kernel 的提交产生数据流量大约在 **1KB**，GPU 执行与 Host 同步的数据量大约在 **0.22KB**，而且从统计结果看与 Kernel 的大小无关，即 **Kernel 的提交开销总体上是固定的**。



不同 Kernel 单次提交产生的数据流量对照图

Reference

CUDA效率优化之：CUDA Graph 改名机会要过期

技术改变AI发展：CUDA Graph优化的底层原理分析（GPU底层技术系列一）–阿里云开发者社区

llm-inference-practice/python/cuda_graph/kernel_commit.py at main · yykzjh/llm-inference-practice